

› Question 1: Loan Eligibility Calculator (Functions)

↳ 2 cells hidden

› Question 2: Function Parameters - Flexible Calculator

↳ 4 cells hidden

› Question 3: Nested Functions - Invoice Calculator

↳ 3 cells hidden

› Question 4: Functions Returning Functions - Currency Converter Factory

↳ 2 cells hidden

✓ Question 5: Lambda Functions - Data Processing

Description

Use lambda functions with `map()`, `filter()`, and `reduce()` to process data.

Requirements

Using `map()`:

- Square all numbers in a list
- Convert temperatures from Celsius to Fahrenheit
- Extract first letter from list of names

Using `filter()`:

- Filter even numbers
- Filter names starting with specific letter
- Filter products above certain price

Using reduce():

- Calculate sum of numbers
- Find maximum number
- Calculate product of numbers

Custom Sorting:

- Sort list of dictionaries by specific key using lambda
- Sort students by grade (descending)

Input/Output Example

Numbers: [1, 2, 3, 4, 5]

Squared: [1, 4, 9, 16, 25]

Temperatures (°C): [0, 25, 37, 100]

Temperatures (°F): [32.0, 77.0, 98.6, 212.0]

Numbers: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Even numbers: [2, 4, 6, 8, 10]

Students sorted by grade: [{'name': 'Alice', 'grade': 95}, {'name': 'Bob', 'grade': 87}, {'name': 'Charlie', 'grade': 78}]

Constraints

- Import functools for reduce()
- Use lambda functions, not regular functions
- Handle empty lists appropriately

```
from functools import reduce

print("=== Lambda Functions with map(), filter(), reduce() ===\n")

numbers = [1, 2, 3, 4, 5]

squared = list(map(lambda x: x**2, numbers))

print("Numbers:", numbers)
print("Squared:", squared)

temperatures_c = [0, 25, 37, 100]

temperatures_f = list(
    map(lambda c: (c * 9/5) + 32, temperatures_c)
)
```

```
print("\nTemperatures (°C):", temperatures_c)
print("Temperatures (°F):", temperatures_f)

names = ["Alice", "Bob", "Charlie", "David"]

first_letters = list(
    map(lambda name: name[0], names)
)

print("\nNames:", names)
print("First Letters:", first_letters)
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

even_numbers = list(
    filter(lambda x: x % 2 == 0, numbers)
)

print("\nNumbers:", numbers)
print("Even Numbers:", even_numbers)

names = ["Alice", "Andrew", "Bob", "Charlie", "Alex"]

a_names = list(
    filter(lambda name: name.startswith("A"), names)
)

print("\nNames:", names)
print("Names Starting with A:", a_names)

products = [
    {"name": "Laptop", "price": 50000},
    {"name": "Mouse", "price": 500},
    {"name": "Keyboard", "price": 1500},
    {"name": "Monitor", "price": 12000}
]

expensive_products = list(
    filter(lambda product: product["price"] > 1000, products)
)

print("\nProducts Above Rs 1000:")
print(expensive_products)

numbers = [1, 2, 3, 4, 5]

total = reduce(
    lambda x, y: x + y,
    numbers
)

maximum = reduce(
```

```

        lambda x, y: x if x > y else y,
        numbers
    )

product = reduce(
    lambda x, y: x * y,
    numbers
)

print("\nNumbers:", numbers)
print("Sum:", total)
print("Maximum:", maximum)
print("Product:", product)

students = [
    {"name": "Alice", "grade": 95},
    {"name": "Bob", "grade": 87},
    {"name": "Charlie", "grade": 78},
    {"name": "David", "grade": 91}
]

students_sorted = sorted(
    students,
    key=lambda student: student["grade"],
    reverse=True
)

print("\nStudents Sorted by Grade (Descending):")
print(students_sorted)
```

=== Lambda Functions with map(), filter(), reduce() ===

```
Numbers: [1, 2, 3, 4, 5]
Squared: [1, 4, 9, 16, 25]
```

```
Temperatures (°C): [0, 25, 37, 100]
Temperatures (°F): [32.0, 77.0, 98.6, 212.0]
```

```
Names: ['Alice', 'Bob', 'Charlie', 'David']
First Letters: ['A', 'B', 'C', 'D']
```

```
Numbers: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Even Numbers: [2, 4, 6, 8, 10]
```

```
Names: ['Alice', 'Andrew', 'Bob', 'Charlie', 'Alex']
Names Starting with A: ['Alice', 'Andrew', 'Alex']
```

```
Products Above Rs 1000:
[{'name': 'Laptop', 'price': 50000}, {'name': 'Keyboard', 'price': 1500}, {'name': 'Mouse', 'price': 1000}, {'name': 'Headset', 'price': 2000}, {'name': 'Webcam', 'price': 1500}, {'name': 'Monitor', 'price': 3000}, {'name': 'Printer', 'price': 1000}, {'name': 'Scanner', 'price': 1000}, {'name': 'Router', 'price': 1000}, {'name': 'Smartwatch', 'price': 1000}, {'name': 'Smart TV', 'price': 2000}, {'name': 'Sound System', 'price': 1000}, {'name': 'Gaming Console', 'price': 3000}, {'name': 'Digital Camera', 'price': 1000}, {'name': 'Smart Home Device', 'price': 1000}, {'name': 'Fitness Tracker', 'price': 1000}, {'name': 'Smart Lock', 'price': 1000}, {'name': 'Smart Doorbell', 'price': 1000}, {'name': 'Smart Light Bulb', 'price': 1000}, {'name': 'Smart Plug', 'price': 1000}, {'name': 'Smart Thermostat', 'price': 1000}, {'name': 'Smart Smoke Detector', 'price': 1000}, {'name': 'Smart Water Leak Detector', 'price': 1000}, {'name': 'Smart Pet Feeder', 'price': 1000}, {'name': 'Smart Dog Collar', 'price': 1000}, {'name': 'Smart Cat Litter Box', 'price': 1000}, {'name': 'Smart Plant Pot', 'price': 1000}, {'name': 'Smart Garden Watering System', 'price': 1000}, {'name': 'Smart Irrigation Controller', 'price': 1000}, {'name': 'Smart Lawn Mower', 'price': 1000}, {'name': 'Smart Hedge Trimmer', 'price': 1000}, {'name': 'Smart Leaf Blower', 'price': 1000}, {'name': 'Smart Chainsaw', 'price': 1000}, {'name': 'Smart Pruner', 'price': 1000}, {'name': 'Smart Shovel', 'price': 1000}, {'name': 'Smart Rake', 'price': 1000}, {'name': 'Smart Wheelbarrow', 'price': 1000}, {'name': 'Smart Garden Hose', 'price': 1000}, {'name': 'Smart Nozzle', 'price': 1000}, {'name': 'Smart Sprinkler', 'price': 1000}, {'name': 'Smart Fertilizer Dispenser', 'price': 1000}, {'name': 'Smart Pest Control System', 'price': 1000}, {'name': 'Smart Security Camera', 'price': 1000}, {'name': 'Smart Doorbell Camera', 'price': 1000}, {'name': 'Smart Motion Sensor', 'price': 1000}, {'name': 'Smart Window Sensor', 'price': 1000}, {'name': 'Smart Glass Break Sensor', 'price': 1000}, {'name': 'Smart Smoke Alarm', 'price': 1000}, {'name': 'Smart Carbon Monoxide Detector', 'price': 1000}, {'name': 'Smart Gas Detector', 'price': 1000}, {'name': 'Smart Fire Alarm', 'price': 1000}, {'name': 'Smart Flood Alarm', 'price': 1000}, {'name': 'Smart Freeze Alarm', 'price': 1000}, {'name': 'Smart Leak Alarm', 'price': 1000}, {'name': 'Smart Pet Alarm', 'price': 1000}, {'name': 'Smart Baby Monitor', 'price': 1000}, {'name': 'Smart Night Vision Camera', 'price': 1000}, {'name': 'Smart Two-Way Audio Camera', 'price': 1000}, {'name': 'Smart Motion-Activated Camera', 'price': 1000}, {'name': 'Smart Infrared Camera', 'price': 1000}, {'name': 'Smart Thermal Camera', 'price': 1000}, {'name': 'Smart 360-Degree Camera', 'price': 1000}, {'name': 'Smart 4K Camera', 'price': 1000}, {'name': 'Smart 8MP Camera', 'price': 1000}, {'name': 'Smart 1080P Camera', 'price': 1000}, {'name': 'Smart 720P Camera', 'price': 1000}, {'name': 'Smart 4MP Camera', 'price': 1000}, {'name': 'Smart 5MP Camera', 'price': 1000}, {'name': 'Smart 3MP Camera', 'price': 1000}, {'name': 'Smart 2MP Camera', 'price': 1000}, {'name': 'Smart 1MP Camera', 'price': 1000}, {'name': 'Smart 0.5MP Camera', 'price': 1000}, {'name': 'Smart 0.3MP Camera', 'price': 1000}, {'name': 'Smart 0.2MP Camera', 'price': 1000}, {'name': 'Smart 0.1MP Camera', 'price': 1000}, {'name': 'Smart 0.05MP Camera', 'price': 1000}, {'name': 'Smart 0.02MP Camera', 'price': 1000}, {'name': 'Smart 0.01MP Camera', 'price': 1000}, {'name': 'Smart 0.005MP Camera', 'price': 1000}, {'name': 'Smart 0.002MP Camera', 'price': 1000}, {'name': 'Smart 0.001MP Camera', 'price': 1000}, {'name': 'Smart 0.0005MP Camera', 'price': 1000}, {'name': 'Smart 0.0002MP Camera', 'price': 1000}, {'name': 'Smart 0.0001MP Camera', 'price': 1000}, {'name': 'Smart 0.00005MP Camera', 'price': 1000}, {'name': 'Smart 0.00002MP Camera', 'price': 1000}, {'name': 'Smart 0.00001MP Camera', 'price': 1000}, {'name': 'Smart 0.000005MP Camera', 'price': 1000}, {'name': 'Smart 0.000002MP Camera', 'price': 1000}, {'name': 'Smart 0.000001MP Camera', 'price': 1000}, {'name': 'Smart 0.0000005MP Camera', 'price': 1000}, {'name': 'Smart 0.0000002MP Camera', 'price': 1000}, {'name': 'Smart 0.0000001MP Camera', 'price': 1000}, {'name': 'Smart 0.00000005MP Camera', 'price': 1000}, {'name': 'Smart 0.00000002MP Camera', 'price': 1000}, {'name': 'Smart 0.00000001MP Camera', 'price': 1000}, {'name': 'Smart 0.000000005MP Camera', 'price': 1000}, {'name': 'Smart 0.000000002MP Camera', 'price': 1000}, {'name': 'Smart 0.000000001MP Camera', 'price': 1000}, {'name': 'Smart 0.0000000005MP Camera', 'price': 1000}, {'name': 'Smart 0.0000000002MP Camera', 'price': 1000}, {'name': 'Smart 0.0000000001MP Camera', 'price': 1000}, {'name': 'Smart 0.00000000005MP Camera', 'price': 1000}, {'name': 'Smart 0.00000000002MP Camera', 'price': 1000}, {'name': 'Smart 0.00000000001MP Camera', 'price': 1000}, {'name': 'Smart 0.000000000005MP Camera', 'price': 1000}, {'name': 'Smart 0.000000000002MP Camera', 'price': 1000}, {'name': 'Smart 0.000000000001MP Camera', 'price': 1000}, {'name': 'Smart 0.0000000000005MP Camera', 'price': 1000}, {'name': 'Smart 0.0000000000002MP Camera', 'price': 1000}, {'name': 'Smart 0.0000000000001MP Camera', 'price': 1000}, {'name': 'Smart 0.00000000000005MP Camera', 'price': 1000}, {'name': 'Smart 0.00000000000002MP Camera', 'price': 1000}, {'name': 'Smart 0.00000000000001MP Camera', 'price': 1000}, {'name': 'Smart 0.000000000000005MP Camera', 'price': 1000}, {'name': 'Smart 0.000000000000002MP Camera', 'price': 1000}, {'name': 'Smart 0.000000000000001MP Camera', 'price': 1000}, {'name': 'Smart 0.0000000000000005MP Camera', 'price': 1000}, {'name': 'Smart 0.0000000000000002MP Camera', 'price': 1000}, {'name': 'Smart 0.0000000000000001MP Camera', 'price': 1000}, {'name': 'Smart 0.00000000000000005MP Camera', 'price': 1000}, {'name': 'Smart 0.00000000000000002MP Camera', 'price': 1000}, {'name': 'Smart 0.00000000000000001MP Camera', 'price': 1000}, {'name': 'Smart 0.000000000000000005MP Camera', 'price': 1000}, {'name': 'Smart 0.000000000000000002MP Camera', 'price': 1000}, {'name': 'Smart 0.000000000000000001MP Camera', 'price': 1000}, {'name': 'Smart 0.0000000000000000005MP Camera', 'price': 1000}, {'name': 'Smart 0.0000000000000000002MP Camera', 'price': 1000}, {'name': 'Smart 0.0000000000000000001MP Camera', 'price': 1000}, {'name': 'Smart 0.00000000000000000005MP Camera', 'price': 1000}, {'name': 'Smart 0.00000000000000000002MP Camera', 'price': 1000}, {'name': 'Smart 0.00000000000000000001MP Camera', 'price': 1000}, {'name': 'Smart 0.000000000000000000005MP Camera', 'price': 1000}, {'name': 'Smart 0.000000000000000000002MP Camera', 'price': 1000}, {'name': 'Smart 0.000000000000000000001MP Camera', 'price': 1000}, {'name': 'Smart 0.0000000000000000000005MP Camera', 'price': 1000}, {'name': 'Smart 0.0000000000000000000002MP Camera', 'price': 1000}, {'name': 'Smart 0.0000000000000000000001MP Camera', 'price': 1000}, {'name': 'Smart 0.00000000000000000000005MP Camera', 'price': 1000}, {'name': 'Smart 0.00000000000000000000002MP Camera', 'price': 1000}, {'name': 'Smart 0.00000000000000000000001MP Camera', 'price': 1000}, {'name': 'Smart 0.000000000000000000000005MP Camera', 'price': 1000}, {'name': 'Smart 0.000000000000000000000002MP Camera', 'price': 1000}, {'name': 'Smart 0.000000000000000000000001MP Camera', 'price': 1000}, {'name': 'Smart 0.0000000000000000000000005MP Camera', 'price': 1000}, {'name': 'Smart 0.0000000000000000000000002MP Camera', 'price': 1000}, {'name': 'Smart 0.0000000000000000000000001MP Camera', 'price': 1000}, {'name': 'Smart 0.00000000000000000000000005MP Camera', 'price': 1000}, {'name': 'Smart 0.00000000000000000000000002MP Camera', 'price': 1000}, {'name': 'Smart 0.00000000000000000000000001MP Camera', 'price': 1000}, {'name': 'Smart 0.000000000000000000000000005MP Camera', 'price': 1000}, {'name': 'Smart 0.000000000000000000000000002MP Camera', 'price': 1000}, {'name': 'Smart 0.000000000000000000000000001MP Camera', 'price': 1000}, {'name': 'Smart 0.0000000000000000000000000005MP Camera', 'price': 1000}, {'name': 'Smart 0.000000000
```

```
Numbers: [1, 2, 3, 4, 5]
Sum: 15
Maximum: 5
```

```
Product: 120
```

```
Students Sorted by Grade (Descending):
```

```
[{'name': 'Alice', 'grade': 95}, {'name': 'David', 'grade': 91}, {'name': 'Bob',
```

✓ Question 6: Decorators - Function Enhancement

Description

Create and use decorators to add functionality to functions.

Requirements

Timing Decorator:

- Create @timing_decorator that measures function execution time
- Display time taken in seconds

Logging Decorator:

- Create @log_decorator that logs function calls
- Log function name, arguments, and return value

Validation Decorator:

- Create @validate_input that validates function arguments
- Check if numbers are positive
- Check if strings are non-empty

Retry Decorator:

- Create @retry_decorator(max_attempts) that retries function on failure
- Retry up to max_attempts times

Apply multiple decorators to a single function

Input/Output Example

```
@timing_decorator
```

```
@log_decorator
```

```
def calculate_factorial(n): #implementation
```

```
pass
```

```
calculate_factorial(5)
```

Output:

```
[LOG] Calling function: calculate_factorial
```

[LOG] Arguments: (5,)

[LOG] Return value: 120

[LOG] Function calculate_factorial took 0.0001 seconds to execute.

Constraints

- Use functools.wraps to preserve function metadata
- Handle decorator arguments properly
- Test with various function signature

```
import time
from functools import wraps

def timing_decorator(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start_time = time.time()
        result = func(*args, **kwargs)
        end_time = time.time()
        print(
            f"[LOG] Function {func.__name__} "
            f"took {end_time - start_time:.6f} seconds to execute."
        )
        return result
    return wrapper

def log_decorator(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        print(f"[LOG] Calling function: {func.__name__}")
        print(f"[LOG] Arguments: {args}")
        result = func(*args, **kwargs)
        print(f"[LOG] Return value: {result}")
        return result
    return wrapper

def validate_input(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        for arg in args:
            if isinstance(arg, (int, float)):
                if arg <= 0:
                    raise ValueError(
                        "Numeric arguments must be positive."
                    )
            if isinstance(arg, str):
                if arg.strip() == "":
                    raise ValueError(
```

```

        "String arguments cannot be empty."
    )
    return func(*args, **kwargs)
return wrapper

def retry_decorator(max_attempts):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            attempts = 0
            while attempts < max_attempts:

                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    attempts += 1
                    print(
                        f"[RETRY] Attempt {attempts} failed: {e}"
                    )
                    if attempts == max_attempts:
                        raise

            return wrapper
        return decorator

@timing_decorator
@log_decorator
@validate_input
def calculate_factorial(n):

    result = 1

    for i in range(1, n + 1):
        result *= i

    return result

counter = 0

@retry_decorator(3)
def unstable_function():

    global counter

    counter += 1

    if counter < 3:
        raise Exception("Temporary failure")

    return "Success"

print("\n=== Factorial Example ===")

```

```
print("Factorial:", calculate_factorial(5))
```

```
print("\n=== Retry Example ===")
```

```
=== Factorial Example ===
```

```
[LOG] Calling function: calculate_factorial
```

```
[LOG] Arguments: (5,)
```

```
[LOG] Return value: 120
```

```
[LOG] Function calculate_factorial took 0.000036 seconds to execute.
```

```
Factorial: 120
```

```
=== Retry Example ===
```

```
[RETRY] Attempt 1 failed: Temporary failure
```

```
[RETRY] Attempt 2 failed: Temporary failure
```

```
Success
```

> Question 7: Custom Module Creation

↳ 5 cells hidden

> Question 8: Package Creation and Organization

↳ 10 cells hidden